Evolution 4 · Lecture

MCP I: Building a Server

N×M becomes N+M; hosts, clients, servers; tools, resources, prompts; the stateless 2026 revision; one server, two clients, every call logged

SURVEY | BUILD | GOVERN | DEFEND

CIDM 6333 · Agentic Systems Design and Development
Survey: ChatGPT plugins · OpenAPI tool specs · MCP and its foundation · Build: an MCP server over your tools; inspector; two clients

Sixty minutes, five moves

0:00–0:02

The run of show

1	0:00	**Framing**	N×M integrations become N+M; what a protocol promises that a plugin format did not
2	0:10	**Spine**	Hosts, clients, servers; the three server primitives and who controls each; the wire after 2026-07-28; building the server
3	0:30	**Teardowns**	ChatGPT plugins (2023); OpenAPI-style tool specs; MCP as a system, and the foundation that now holds it
4	0:45	**Reference design**	Why gzkit refused MCP by doctrine; the governance-server project: gz verbs and the ledger as tools and resources
5	0:52	**Evaluation lens**	What a strong Sortie 4 looks like; Flight Card 4 brief (center of envelope)

N×M becomes N+M

0:02–0:10

The problem was combinatorial before it was technical

N hosts and M tools meant N×M adapters. A protocol makes each side implement one thing: N clients, M servers, N+M pieces of code.

Three promises a signature cannot make

Discovery: a client asks what a server offers before calling. Negotiation: versions and capabilities stated; a mismatch is an error. Transport independence: a pipe on your laptop, HTTP on a host you do not control. Your E3 tool had none of these.

Tonight's claim, which Sortie 4 will try to disprove

Every declared tool is discoverable, callable, and schema-validated from two independent clients; every call is logged server-side. Works from the harness but not the script? The harness did the protocol's work, and you have measured it.

Hosts, clients, servers, and the three primitives

0:10–0:16

The architecture, and the question of who controls each primitive

Host · Client · Server

Host: the application the human is in. Client: one connector per server, inside the host. Server: declares capabilities and answers requests over JSON-RPC 2.0.

The human sits in the host. The server never sees the human.

Tools

Functions the model may call: a name, a description, a JSON Schema for input. The model chooses; the host may gate the call.

Model-controlled. E3's contract, on the wire.

Resources

Data the server exposes by URI: files, records, a ledger. The application decides what enters context, and when.

Application-controlled. Content, not instructions (E5).

Prompts

Templates the server offers the user: a named workflow with arguments, surfaced as a slash command.

User-controlled. The human picks; the server proposes.

Extensions and tasks

An `extensions` field in both capability sets; long-running work moved to the tasks extension (`tasks/get`, `tasks/update`). Adopt an extension only when the core cannot express the need.

Negotiated, not assumed.

Deprecated in 2026-07-28

Roots, sampling, and logging: deprecated, twelve-month window. HTTP+SSE transport: deprecated; use Streamable HTTP.

Do not build a new server on a deprecated feature.

The wire after 2026-07-28: a stateless core

0:16–0:22

What changed at general availability, and what it does to the server you build tonight

Before (2025-11-25)	After (2026-07-28)	Consequence for your server
`initialize` handshake, then `notifications/initialized`	No handshake. Every request carries version and capabilities in `_meta`; servers answer `serverInfo` in `_meta`	A request can land on any instance behind a plain load balancer; nothing to remember
`Mcp-Session-Id` header on Streamable HTTP	Removed. `tools/list` no longer varies per connection; cross-call state uses server-minted handles passed as arguments	A session becomes a handle, and the handle is a design decision to record
Server-initiated requests: `roots/list`, `sampling/createMessage`, `elicitation/create`	Multi Round-Trip Requests: server returns `resultType: "input_required"` with `inputRequests`; client retries with `inputResponses`	Ask the human by returning, not by calling back; every result carries `resultType`
Capabilities discovered at handshake	`server/discover` — mandatory RPC advertising versions, capabilities, identity; usable as a back-compat probe on stdio	The inspector's first call; Test Point 1 of Sortie 4
`resources/subscribe`, HTTP GET stream	`subscriptions/listen` — one long-lived POST stream for opted-in list-changed notifications	Optional for the sortie; declare it only if you implement it
List results unbounded	`ttlMs` and `cacheScope` required on list and read results; deterministic tool order recommended	Order your tools; give an honest TTL; the client will cache on it

Source: modelcontextprotocol.io, 2026-07-28 changelog (GA July 28, 2026; Tier 1 SDKs shipped same day). Verify the lab's Python SDK version against this table before class.

Building the server: declare, run, inspect, connect twice

0:22–0:30

FastMCP-style: the schema comes from the type hints; the transport is a flag

Declaration (Python SDK, FastMCP style)

```
from mcp.server.fastmcp import FastMCP
srv = FastMCP("e3-tools")

@srv.tool()
def read_note(path: str) -> str:
    """Read a note. Side effects: none."""
    log("call", "read_note", path)
    return NOTES.read(path)

@srv.resource("ledger://events")
def ledger() -> str:
    return LEDGER.tail(50)

srv.run(transport="stdio")  # HTTP later
```

The four moves of the lab, in order

1. Declare one E3 read tool: docstring as description, type hints as schema.
2. Run over stdio; the inspector confirms server/discover and tools/list.
3. Connect two clients: your harness and a twenty-line script.
4. Read the server log: two calls, two matching entries.

The description is the model's only view of the tool. Write it as a contract: what it does, what it must not do, side-effect class.

Teardown: ChatGPT plugins and OpenAPI-style tool specs

0:30–0:37

Description without negotiation: why the pre-protocol attempts stalled

ChatGPT plugins (March 2023)

A manifest, an OpenAPI document, a paragraph the model read. Vendor-hosted, one product, one store.

Right: a model could read a spec and call an endpoint. Lacked: other hosts, negotiation, local transport, content or prompts.

Control: the vendor owns discovery, hosting, and the loop.

OpenAPI-style tool specs (2023–)

Function calling, mid-2023: a JSON Schema per function, a structured call back; schemas generated from OpenAPI.

Right: schemas as contracts, in-process, any model. Lacked: discovery, negotiation, a second primitive. Each host wrote its own loader.

Control: you own loop and schema; nobody owns the boundary.

MCP as a system: where is the human?

0:37–0:45

November 2024 to the foundation; a Design Analysis on the board

Timeline

Nov 2024 · protocol, SDKs, reference servers

Mar 2025 · Streamable HTTP; OAuth authorization

Jun 2025 · resource indicators, structured output, elicitation

Nov 2025 · tasks (experimental), URL-mode elicitation

Dec 2025 · donated to the Agentic AI Foundation

Jul 2026 · GA 2026-07-28: stateless core

Where is the human?

In the host, and only in the host. The client must get the human's approval before a tool runs; the server never assumes it. The wire carries requests, not permissions.

Governance is delegated to the host by design; E5 shows that trust misplaced.

What a healthy protocol ecosystem needs

A versioning and deprecation policy (twelve-month window). Independent SDKs (TypeScript, Python, Go, C#; Rust in beta). A server registry with provenance, or it is a supply chain (E5: SmartLoader). Governance that is not one vendor.

Read it as a Design Analysis

Problem: N×M. Decision: a client–server protocol over JSON-RPC, three server primitives, negotiated capabilities. Alternatives: a plugin store; per-vendor function calling; a library. Consequences: adoption across hosts; a new security surface. The human: outside the wire.

Reference design: why gzkit refused MCP

0:45–0:49

storage-tiers.md — 'no external protocol may become a prerequisite for core governance operations'

The doctrine and its reasoning

The verbs (gz status, gz validate, ledger, receipts) run on the standard library over files.

Why: attestation must not depend on availability. The ledger must be readable with cat. Same posture as ADR-0.0.2.

Campaign review: MCP 'absent and un-designed', but the refusal was written down.

What changes one tier up

Prerequisites, not presence. A server above the core adds a way in, not a dependency.

The pool ADR sketches this tier. Falsifier: kill the server, run gz validate.

Reference design: the governance server as a student project

0:49–0:52

gz verbs as tools, the ledger as a resource, 'what is attested?' answered from the record

Tools (model-controlled)

gz_status(): the repo's governance state. gz_validate(scope): run the validators, return the report. Read-class only. Write verbs (gz attest, gz release) stay behind a host gate, as in E3.

Resources (application-controlled)

ledger://events?tail=N: the last N events. attest://<artifact>: what is attested for one artifact. receipts://<id>. The host decides when these enter context.

Prompts (user-controlled)

/what-is-attested: a template that reads the attestation resource and answers only from it. The answer is checkable against the resource it was given.

Chapter 13 exercise: ask your harness 'what is attested in this repo?' and check the answer against the ledger. An added attestation is summary laundering, and the server is the instrument to catch it.

Evaluation lens: a strong Sortie 4

0:52–0:56

The Chase reads two client logs and one server log, and nothing else

Weak

- One harness plays both clients, prompted twice
- tools/list schema differs from the function; the client 'fixed' the arguments
- Server log missing or client-written; calls counted from the narrative
- A write tool on the wire, no gate
- Built on roots or sampling from a 2025 tutorial

Sound

- server/discover and tools/list captured; schema matches type hints
- Two clients (harness + script), same call, same result, both logged
- Server log: timestamp, tool, arguments, outcome
- Read tool on the wire; write tool held back, reason recorded
- SDK version and protocol revision on the SUBSTRATE line

Exceptional

- A third client, another harness family, divergence explained
- A wrong schema on purpose; the server rejects it
- ttlMs and cacheScope set; a client observed honoring them
- Governance server: gz_status, ledger resource, 'what is attested?' diff
- Debrief: what the protocol forced that E3 let you skip

Flight Card 4 brief: one server, two clients

0:56–1:00

Center of the envelope · depends on Sortie 3 PASS

```
ENVELOPE            center
DESIGN CLAIM        Every tool the server declares is discoverable, callable, and schema-validated from two
                    independent clients, and every call is logged server-side.
SYSTEMS UNDER TEST  E3 read tool · MCP server · inspector · harness client · script client · server log · ledger
SUBSTRATE           repo + commit · Python SDK version · protocol revision claimed · model/route · stdio
DEPENDS ON          Sortie 3 PASS (gate on the write tool holds)

TEST POINTS
  1  inspector → server/discover, tools/list          → versions, capabilities, one tool with the intended schema
  2  harness client → tools/call read_note(path)      → result + server log entry {ts, tool, args, outcome}
  3  script client  → tools/call read_note(path)      → identical result + second log entry
  4  script client  → tools/call with a bad argument  → server rejects on schema; log shows the rejection

INSTRUMENTATION     server log (4 entries, in order) · inspector capture · both client transcripts · ledger event

PRE-REGISTERED FALSIFIERS (any ONE ⇒ FAIL)
  - a tool callable from one client but not the other
  - a call with no server-side log entry
  - a schema the client accepted that the server rejected, or the reverse
  - passed only by disabling validation or by editing the log

ABORT (halt-not-fail)  SDK/protocol-revision mismatch that prevents discovery; model unavailable
CHASE               reads the four log entries and the inspector capture; never the flight-log narrative
```

A protocol is a contract that strangers can discover.

Put one tool on the wire, let two clients find it, and let the server keep the only log that counts. Consultation: Sortie 3 verdicts and your server's first tool in one sentence.